Project 2

Effect of Associativity on Cache Performance

CSCE2303 — Assembly and Computer Organization

Dr.Mohamed Shalan

Team Members

Jana Hassan
Sedra Elkhayat
Sama Almallak

July 19, 2026

1 Cache Implementation

Fixed configuration: 64 KB cache, 64 B lines, associativity swept over 1, 2, 4, 8, 16 ways, random replacement, write-back with write-allocate.

1.1 Data Structures

The cache is a two-dimensional vector, `cache[set][way]`, where each element is:

```
struct CacheLine {
    bool valid;
    bool dirty;
    unsigned int tag;
};
```

`initCache(ways)` resets the write-back counter, reseeds the replacement RNG for reproducibility, computes

$$\text{num\_sets} = \frac{\text{CACHE\_SIZE}}{\text{LINE\_SIZE} \times \text{ways}},$$

and allocates a cold (`valid=false`) grid of that size.

Ways	Sets
1	1024
2	512
4	256
8	128
16	64

Table 1: Number of sets for each associativity (total lines fixed at 1024).

1.2 Address Breakdown

Each address splits into tag, set index, and offset. With a 64 B line, the offset is the low 6 bits; the remaining fields are computed as:

```
blockAddr = addr / LINE_SIZE;
setIndex = blockAddr % g_numSets;
tag = blockAddr / g_numSets;
```

1.3 Hit and Miss Path

Hit: all `ways` lines in the target set are scanned for `valid && tag == tag`; a match returns `HIT` and sets `dirty=true` on a write, with no replacement needed.

Miss, empty slot: if any line in the set is invalid, the new block is inserted there (`valid=true`, `dirty` = is-write), no eviction required.

Miss, set full: a victim is chosen via `randReplace() % g_ways` (an RNG stream independent from the address generators, so associativity never changes the request sequence itself). If the victim was dirty, the write-back counter increments before it is overwritten.

1.4 Write Policy

Write-back: writes only mark the line dirty (`line.dirty = true`); main memory is updated only at eviction. Write-allocate: a write miss loads the block into the cache before marking it dirty, rather than writing through to memory directly.

2 Experimental Setup

All experiments used the fixed configuration above over a 64 MB simulated DRAM space (word-aligned, 4 B accesses), with 1,000,000 accesses per run and 30 total runs (6 generators × 5 associativities). Before each run, both the generator (`resetMemGens()`) and the cache (`initCache()`) were reset, so every associativity value saw the identical request sequence — any change in hit ratio is only due to associativity.

Parameter	Value
Cache size	64 KB
Line size	64 B
DRAM size	64 MB
Associativity (sweep)	1, 2, 4, 8, 16 ways
Replacement policy	Random (`randReplace()`)
Write policy	Write-back + write-allocate
Accesses per run	1,000,000
Total runs	30

Table 2: Fixed experimental parameters.

Generator	Pattern	Design intent
memGen1	2 addresses, same set	Minimal conflict, resolves at 2-way
memGen2	Uniform-random, full 64 MB	No locality; capacity limited
memGen3	8 addresses, same set	Larger conflict, resolves at 8-way
memGen4	16 sets × 4-way conflict	Program-like, resolves at 4-way
memGen5	Sequential word scan	Spatial locality only
memGen6	Stack push/pop, ≤16 KB	Tiny working set

Table 3: Generator patterns and design intent.

2.1 Results

Generator	1-way	2-way	4-way	8-way	16-way	Ways to saturate
memGen1	0.00%	100.00%	100.00%	100.00%	100.00%	2
memGen2	0.10%	0.10%	0.10%	0.10%	0.10%	never (capacity)
memGen3	0.00%	0.78%	17.67%	100.00%	100.00%	8
memGen4	0.00%	14.28%	99.99%	99.99%	99.99%	4
memGen5	93.75%	93.75%	93.75%	93.75%	93.75%	n/a (locality)
memGen6	99.99%	99.99%	99.99%	99.99%	99.99%	n/a (fits at ways=1)

Table 4: Hit ratio vs. associativity, all six generators.

Generator	1-way	2-way	4-way	8-way	16-way
memGen1	300,776	0	0	0	0
memGen2	499,036	499,048	499,055	499,056	499,043
memGen3	300,776	300,083	282,465	0	0
memGen4	300,770	286,598	0	0	0
memGen5	50,189	50,199	50,204	50,200	50,198
memGen6	0	0	0	0	0

Table 5: Write-back counts vs. associativity.

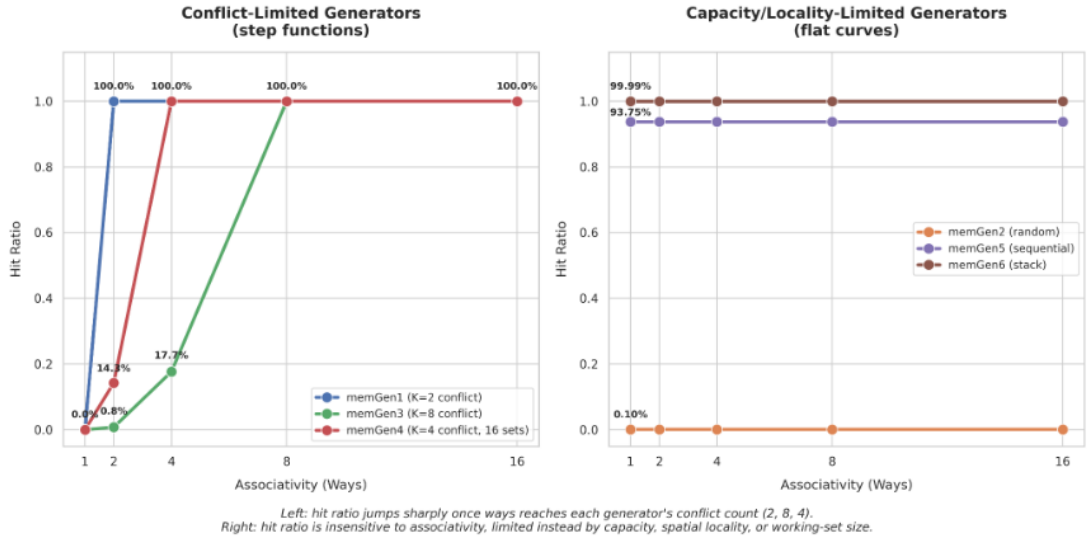

Figure 1: Side-by-side analysis isolating conflict-limited stepping behaviors (left) from locality-limited flat trends (right)

2.2 Verifications

Manual trace: We hand-traced `memGen1`'s first four accesses (addresses 0 and 65536, both mapping to set 0). At ways=1, the single line is evicted every access, so all four miss; at ways=2, both addresses get separate lines after two cold misses, so accesses 3–4 hit. This matches the simulator's aggregate output exactly (0.00% vs. 99.9998%), confirming eviction and tag-matching logic at the individual-access level.

Completeness: reads + writes = 1,000,000 for all 30 runs.

RNG verification: Measured write-fraction matched each generator's target (e.g. 30.08% vs. 30% target for memGen1/3/4; 49.98% vs. 50% for memGen2; 10.04% vs. 10% for memGen5). The actual workloads align precisely with expected sampling variations over a 1,000,000-draw distribution.

Write-back / hit-ratio cross-check: write-backs collapse to exactly 0 the instant a conflict-driven generator saturates to ≈100% hit ratio, confirming dirty-bit and eviction logic are internally consistent.

Random-replacement nuance: memGen3 climbs gradually (0.78% at 2-way, 17.67% at 4-way) rather than flat-lining at 0% until ways=8 as a deterministic LRU policy would — confirming random replacement occasionally spares a needed line by chance.

3 Analysis

Across the six generators, associativity separates cleanly into two behaviors: **conflict-limited** generators, whose hit ratio steps sharply upward at a specific ways threshold, and **capacity/locality-limited** generators, flat regardless of associativity; mapping directly onto the classic three C's of cache misses.

Conflict misses (memGen1, memGen3, memGen4). Each generator aliases a small group of addresses to the same set (via a `CACHE_SIZE` offset that preserves set index across every ways value tested). Below the conflict count, the set has fewer slots than competing addresses, so each access evicts the line the next access needs. Once ways reaches that count, all addresses fit simultaneously and the conflict vanishes: memGen1 (K=2) saturates at ways=2, memGen3 (K=8) at ways=8, memGen4 (16 sets of K=4) at ways=4. These are true conflict misses, the data fits in total cache capacity, but poor placement forces repeated eviction of soon-to-be-reused lines. memGen3's gradual climb below saturation (rather than a flat 0%) reflects random replacement occasionally sparing a needed line by chance, unlike deterministic LRU which would thrash completely until the exact resolving associativity.

Capacity misses (memGen2). Addresses spread uniformly over 64 MB give the cache no spatial or temporal locality to exploit, regardless of how its 1024 lines are grouped into sets and ways. Hit ratio stays pinned near 0.10% across the entire sweep, and write-backs stay flat near 499,000 at every ways value, the clearest demonstration that associativity fixes *placement*, not *capacity*: reorganizing a fixed number of lines cannot compensate for a working set thousands of times larger than the cache.

No conflict pressure (memGen5, memGen6). memGen5's sequential scan benefits from spatial locality alone: each 64 B line satisfies 16 consecutive word accesses, giving a fixed 93.75% hit ratio independent of associativity, since there is no competition for any set. memGen6's $\leq$16 KB stack working set is small enough relative to the 64 KB cache that it never overflows a set even at ways=1, hitting near 100% throughout.

Diminishing returns. Each conflict-limited generator needs exactly as many ways as it has conflicting addresses, and no more, memGen1 and memGen3 gain nothing moving beyond their resolving associativity (2-way and 8-way respectively), both already saturated at 100%. Write-backs collapse to exactly 0 the moment each generator saturates, since an all-hits workload never evicts anything. Beyond the resolving ways value, additional associativity is pure overhead — comparison logic and hardware cost with no hit-ratio benefit for that workload.

4 Conclusion

Associativity is a targeted fix for conflict misses: it helps precisely when multiple frequently-reused addresses collide on the same set, in direct proportion to how many addresses are colliding. It provides no benefit against capacity misses, where the working set exceeds the cache regardless of internal organization, and it is irrelevant when a workload's locality or working-set size already keeps it conflict-free. Choosing an associativity in practice therefore depends on the conflict behavior a real workload is expected to produce — over-provisioning ways beyond what that behavior requires buys no additional hit-ratio benefit for real hardware cost.